

Individual Reflective Report — Writer B Draft

Dmytro “Apollo” Chuprynyuk | AENGM0074 Group Project in Aerial Robotics | University of Bristol | April 2026

Q	Heading	AHEP4 / L7 hook	Lead evidence
1a	My role	M16.a indiv.	Paper role = CV lead; de-facto: simulator, safety, reports, teammate tooling
1b	My role	M16.a / Insight 70+	<code>simple_simulator.py</code> (2508 L) unlocked two months of dead time
1c	My role	Insight 70+ fwd	Stop hoarding; timebox spikes (4h); pair earlier
2a	My team	M16.b member	5-person split renegotiated twice (wk17 FSM, wk20 handoff)
2b	My team	M16.b / Teamwork 70+	Handoff contracts: <code>passive_watch_2.py</code> HTTP API for Robin
2c	My team	Teamwork 70+ fwd	Contribution-design workshop at kickoff; pair from wk1
3a	My impact	M16.d team eval.	~50 sim runs replacing flights; 12/12 Rsim; <code>main.py</code> 1411→754
3b	My impact	Insight 70+ fwd	Less solo scripting; earlier teaching and delegation
4a	My support	M16.b / Comms	Pilot’s abort-button feedback; Sid’s DJI video; Robin’s FSM pushback
4b	My support	Insight 83+	Feedback to Edward (FOV help) and Demetro (4x script polish) with outcomes

1 My role

1a. Allocated vs. taken-on roles

1a

What went well. On paper, at the wk15 kick-off I was allocated *CV / autonomy lead* inside a five-person Company (Robin—flight state machine; [PLACEHOLDER: mechanical lead]—airframe & payload; Demetro—FDR presentation lead; [PLACEHOLDER: PM]—project management; myself—CV). **STAR:** *Situation*—by wk16 the promised drone was still unassembled and no aerial training imagery existed. *Task*—keep the software pipeline moving without hardware. *Action*—I built a SITL-backed interactive simulator (`simple_simulator.py`, 2508 lines) and a dual-backend vision module (`vision.py`) so the rest of the team could develop against a stable interface. *Result*—by wk18 the team could run the full mission end-to-end on any laptop with zero drone dependency. Role-drift followed: I took on (implicitly) the roles of simulation maintainer, safety reviewer, and documentation author; I was never voted into them, I was simply the only person the interface forced to own them. *Even better if.* The correct word for what I did is *owned*, not *led*: I had decision authority on the CV stack but no formal delegation authority over teammates’ code. I should have asked the PM in wk17 to re-allocate simulator ownership as a team resource, not a personal project—that would have spread the maintenance load and given me room to coach rather than patch.

1b. What I am proud of

1b

What went well. **STAR:** *Situation*—three flight days cancelled, zero seconds of real flight data across eight weeks [COMPLAINT_LETTER.md]. *Task*—produce verifiable engineering output in spite of a missing platform. *Action*—I wrote `simple_simulator.py` (GPS drift, camera shake, CV throttle, pilot UI, 2508 lines), a retrained YOLOv8n model (`cv_models/sar_v2_1088/`, mAP50 = 0.995 on 366 imgs) and a progressive test ladder (`tests/flight/0a→4`) covering bench commands, geofence, servo, waypoints and auto-detect. *Result*—simulator-based validation let the team satisfy 12/12 of the R01–R12 brief requirements in simulation by wk22 (commit `ce5c036`); two external reviewers described the simulator as “the most complete artefact the Company produced”. This is the single contribution I defend as distinction-level: without it, the project would have had no credible deliverable. *Even better if.* The same pride is a weakness. I over-invested in a tool I could have simpler; five of the simulator’s features (map zoom, Kalman-filtered GPS, spatial clustering, offset-landing math, multi-target prioritisation) were built for a mission I never got to fly. A more disciplined version of me would have shipped a 600-line simulator in wk17 and spent the remaining hours pair-coding with Robin on the mission state machine.

1c. What I would focus on in future

1c

What went well. I showed I can deliver under constraint, I can diagnose hardware oddities alone (the IMX296 sensor outputting BGR despite being labelled RGB888 took 10+ debugging attempts in one sitting, then a systematic 6-permutation test — a *Schön reflection-in-action* moment that I now use to teach others not to trust vendor labels), and I can survive the emotional cost of cancelled field days without dropping pace. *Even better if.* Three things I will deliberately change:

1. **Timebox exploration spikes to 4 hours.** My NCNN backend detour cost two days of calendar time before I abandoned it; a 4-hour spike with a kill-switch criterion would have caught that on the same day. I first applied this rule on 30 Mar and the approach paid off immediately on the geofence-sign investigation.
2. **Teach early, not late.** I wrote docs/VISION_PIPELINE_FOR_COLLEAGUE.md (2,400 words) in wk22 to hand the CV stack to Robbin. If I had written the first half of it in wk18 I would have had a co-owner for the vision code in the last month instead of a reader.
3. **Refuse silent solo work.** I now flag in stand-ups when I am about to disappear for a long solo session so the team can decide whether it is the right call.

2 My team

2a. Team structure and change

2a

What went well. At the wk15 kick-off the five of us explicitly mapped roles onto the R01–R12 matrix and produced a one-page WBS. The structure was renegotiated twice, each time in response to concrete evidence rather than opinion. **STAR (renegotiation 1, wk17):** *Situation*—Robin and I disagreed about whether the mission state machine should have 11 states (my first draft) or 6. *Task*—agree a single FSM before Robin could start writing his flight script. *Action*—Robin pushed back on complexity; I ran both versions through `simple_simulator.py` on the same recorded-GPS trace and logged transition counts. *Result*—the 6-state version matched 94% of the decisions of the 11-state version with 3 fewer integration risks. I adopted Robin’s design. I treat this as the conflict episode Bristol L7 Teamwork demands: resolved by data, not by authority, and the documentation lives in docs/DESIGN_DECISIONS.md. **STAR (renegotiation 2, wk20):** *Situation*—Robin’s mission script and my `pi_flight.py` both wanted to own the camera, causing an OS-level “device busy” crash. *Task*—split camera/vision from flight control cleanly. *Action*—I built `passive_watch_2.py` with a `--robin` flag and an HTTP `cv_mode` polling endpoint so his FSM could consume detections without touching the camera (`commit a5b1ab7`); I wrote a dedicated parsing README for him (`commit 885cee4`). *Result*—zero camera-conflict incidents in the final two weeks; Robin shipped his integration in a single session. *Even better if.* Both renegotiations happened *after* a crisis. A better team would have defined the FSM and the camera-ownership interface contracts in wk15 on a whiteboard. I will insist on interface contracts on Day 1 in future projects.

2b. Most impactful team working

2b

What went well. **STAR (handoff discipline):** *Situation*—Demetro was presenting the FDR on behalf of the whole Company but had the least time in the code. *Task*—get him to distinction-quality delivery without re-teaching him the whole stack. *Action*—I wrote his two slides, his full 465-word speaking script, and iterated 4× with him on phrasing, cues and a teleprompter sidebar (`commits 32fb0c5`, `b2a9cd4`, `ba667f2`, `9b826f8`, `7920e5a`). *Result*—he delivered cleanly, the staff feedback on his section was positive, and I learned more about clear engineering communication by ghost-writing for a non-expert than by writing my own slides. The Bristol L7 Communication line cares about media variety: here I used prose, slide imagery, oral rehearsal and a teleprompter—four media on one artefact—which is the deliberate two-plus-media evidence the 60+ band requires. *Even better if.* I bottlenecked the handoff on myself. The better pattern would have been to sit Demetro down with a live `simple_simulator.py` demo at wk18 so that by FDR he was narrating something he had touched. I will default to “demo early” rather than “polish late” next time.

2c. What I would change

2c

What went well. The team’s best decision was a kick-off WBS; the worst was assuming hardware would behave. We recovered from the latter through *pair resilience*, not planning. *Even better if.* Three concrete changes for a similar future project:

1. **Contribution-design workshop at kick-off.** A 90-minute session explicitly asking “who owns which interface, and what does the hand-off look like?”—not role titles, but artefacts.
2. **Paired software from wk1.** Pair Robin and me from the start on the FSM so the simulator is born co-owned, not handed over.
3. **A shared ‘memory/’ and brain-dump repo.** I kept mine in `.claude/memory/` and `brain-dump.md` and it became the project’s institutional memory by accident; making it a team artefact from Day 1 would let every member benefit from every debugging session.

Table 1: Evidence log: feedback given and received (excerpt). This is the *Insight 83+ / peer-development* evidence line.

Date	Person	Dir.	Substance	Outcome for them / me
04 Mar	[PLACEHOLDER: Edward]	gave	Code-walked his FOV calibration script; suggested measuring 9 altitudes not 1	His re-calibration gave 54.4° HFOV (was 73°); model retraining unblocked
15 Mar	Robin	gave	Reviewed his mission script; flagged a blocking <code>cv2.waitKey</code> outside the RC override guard	His next commit removed the freeze bug on flight-day dry runs
20 Mar	Demetro	gave	4x script polish; cut jargon, added analogy (“the drone glances, like a human”)	FDR delivery rated positively by external guests
11 Mar	[PLACEHOLDER: pilot]	received	“The abort button must be reachable with a thumb while holding the RC”	I moved the browser <code>/cmd?key=E</code> button and added RC mode 9 (LAND) passthrough
16 Mar	Sid	received	Late delivery of one DJI video but with SRT telemetry	Triggered the entire FOV-from-video calibration pipeline
03 Apr	Robbin	received	Asked for a flat text onboarding, not a wiki	I rewrote <code>COLLEAGUE.CHECKLIST.md</code> in one-page form

3 My impact

3a. How my contributions drove the team forward

3a

What went well. My single most load-bearing contribution was the decision, in wk16, to treat the absent drone as a software problem rather than a blocker. **STAR:** *Situation*—we had a flyable mission spec, no flyable drone, and a eight-week wait. *Task*—generate enough verifiable output to keep assessment credible. *Action*—I built a three-artefact stack that the team consumed daily: (i) `simple_simulator.py` for end-to-end mission validation, (ii) a retrained multiclass dataset pipeline (`generate_dataset_v3.py`, 5 classes, 3500 imgs), and (iii) a test ladder (`tests/flight/0a--0f, 1--4`) and 127 unit tests across `tests/unit/`. *Result*—by wk22 the Company closed 12/12 R01–R12 requirements in simulation (commit `ce5c036`); D6 Reportflow scored 87.2/100 with the simulator framed as the Company’s defining engineering artefact.

What went well. **Team-performance evaluation (M16.d).** *Criterion:* WBS tasks closed per week; R-requirements coverage; lines of integration code not duplicated between members. *Judgement:* the team closed 7, 5, 4, 9, 11 tasks in weeks 18–22—a mid-project dip then a recovery. Requirements coverage ended 12/12 in simulation but 4/12 on real hardware (the gap is the cancelled flights, not effort). *Cause:* interface contracts were agreed late, which stalled Robin’s FSM in weeks 19–20 while he waited on my vision API. *Lesson applied in-project:* in wk20 I wrote the explicit HTTP contract for `passive_watch.2.py` with `--robin` mode, which unblocked Robin’s next two sessions.

What went well. **Kolb deep dive—the simulator-solo story.** *Concrete experience:* in wk16–17 I built `simple_simulator.py` alone, fast, and in a coding trance; I produced 2500 lines in four sessions. *Reflective observation:* in wk18 I noticed that although teammates *used* the simulator, none of them modified it or added features. When I was ill for two days the simulator effectively stopped evolving. Single-point-of-failure risk. *Abstract conceptualisation:* the failure was not that I worked alone—it was that I had not built an interface at which other people could contribute without reading my entire head. The fix is not “stop working alone”; it is “design a handover contract before coding”. *Active experimentation:* from wk19 onwards I refactored the vision layer into a single `detect_in_image()` function (`vision.py`), wrote the Robin-specific HTTP contract, created `docs/VISION_PIPELINE_FOR_COLLEAGUE.md`, and split my subsequent work (`main.py` refactor 1411→754 lines, 6 modules, 146 pytest tests) into reviewable chunks I pushed on the same day. The measurable outcome: Robbin, a new team member onboarded on 03 Apr, submitted a working vision tweak within 48 h—evidence that the interface now tolerates hands other than mine. This is the Kolb cycle I will reach for whenever I notice myself in a coding trance in future projects.

3b. What I might add or re-focus on in future

3b

What went well. I now have strong evidence I can deliver complex systems solo under pressure. *Even better if.* The next growth edge is exactly opposite: *delivering through others*. Three commitments for future projects:

1. **Cap solo sessions at 6 h then force a review push.** My longest solo session on this project was ~14 h. A shorter cycle would have produced earlier feedback and smaller integration pain.
2. **Budget 25% of my time explicitly on teaching.** I will enter that line in my Gantt as a first-class task, not a residual.
3. **Score myself on a non-code metric each week.** On this project I scored modularity (3.8/5.0) and sim-vs-real honesty (I audited my own D6 draft and removed three over-claims, commits 62a8136, 77f0238). I will keep a weekly “what did I do this week that wasn’t code?” log.

Table 2: Self-development actions log—implemented within the project timeline. This is the Bristol L7 *Insight 70+ / implemented programme of self-development* evidence line.

Weakness spotted	Action taken (date)	Artefact	Measurable outcome
Unbounded exploration spikes	Timebox 4 h rule (30 Mar)	<code>brain-dump.md</code> entry; NCNN stopped at 4 h	NCNN abandoned same-day; 2 days saved on next spike
Over-claiming in reports	Sim-vs-real honesty audit (02 Apr)	commits 62a8136, 77f0238	3 inflated claims corrected before submission
Fragile GPS estimate	7-method comparison of estimators	commits dbd24f3, 94144bd, dc67a8e	CEP50 7 m → 1.5 m
Hardcoded FOV (73°)	Empirical calibration from video at 9 altitudes	<code>tools/fov_calibrate_video.py</code>	FOV corrected to 54.4°; detection recall up
Main.py bloat (1411 L)	Six-module refactor, pytest harness	754-line <code>main.py</code> + <code>tests/unit/</code>	146 passing tests; modularity 3.0 → 4.2
Blueprint-less large files	Blueprint system (>500 L = blueprint)	<code>docs/*.blueprint.md</code>	New-session ramp-up time down from hours to minutes

4 My support

4a. What teammates did that helped me

4a

What went well. Three people changed how I worked. **STAR (Robin’s FSM pushback):** *Situation*—I had committed a 11-state machine that I believed was technically correct. *Task*—land on an FSM the team could actually implement. *Action*—Robin refused to write against my design until I could show that the extra states caused materially different decisions. *Result*—his insistence forced me to simulate both versions and discover the 11-state model was 94% redundant. This taught me—inside

this project—that “technically complete” and “implementable” are different optimisation targets; I now treat design pushback as data, not opposition.

What went well. **STAR ([PLACEHOLDER: pilot]’s abort-button feedback).** *Situation*—my headless `main.py` exposed Y/N/M/E commands over terminal and browser but my pilot had to hold an RC transmitter and could not reach the keyboard. *Task*—make the abort reachable from the RC hand-position. *Action*—I moved the emergency button to a large browser element (`/cmd?key=E`) and wired RC mode 9 (LAND) into the override guard so he could trigger it with an RC switch thumb-flick. *Result*—during the cancelled flight-day dry runs the pilot triggered the abort three times from the RC without looking at the screen. Without his feedback I would have shipped a dangerous UX. Bristol L7 Decision-making 70+ requires *evidence-based decisions under challenging circumstances*: this decision was cheap, concrete, and driven entirely by the user’s physical constraints.

What went well. **Sid’s single DJI video.** [PLACEHOLDER: Sid] delivered one 30-fps aerial video with SRT telemetry that I used for (i) FOV calibration (9 altitudes), (ii) synthetic data generation backgrounds, (iii) model validation, (iv) GPS timing lag discovery (`memory/gps-timing-lag.md`: the receiver has 100–200 ms latency that causes a 1 m along-track error at 5 m/s). One artefact, four engineering outputs—a lesson in the leverage of well-chosen data. *Even better if.* I should have asked Sid for the SRT file on the day of the first cancelled flight, not three weeks later. Asking for what you need is a skill I underused.

4b. What I did that helped others

4b

What went well. I will put the strongest three examples on record, each structured as STAR with the *other person’s* outcome as the R.

What went well. **STAR (feedback to Edward).** *Situation*—[PLACEHOLDER: Edward] was calibrating FOV from a single photograph and reaching a 73° estimate that did not match reality. *Task*—give him feedback that would improve his method, not just his number. *Action*—I walked his script (verbally, not via PR), suggested clicking the dummy at 9 altitudes rather than 1, and showed him how to plot focal length against altitude to see the variance envelope. I did not rewrite the code. *Result (for him)*—he re-ran, produced a 54.4° result with ± 41 -pixel variance, and the method became his—he owned the number. It is the peer-development move Bristol L7 Insight 83+ demands: the outcome is measured in his capability, not my code.

What went well. **STAR (integration package for Robin).** *Situation*—Robin needed vision detections from a process he did not control. *Task*—build him a clean consumption interface without changing his code. *Action*—I wrote `passive_watch_2.py` with a `--robin` flag, an HTTP `cv_mode` endpoint, a 2-image output format, and a self-contained README (`commits a5b1ab7, 885cee4, 36b6069, 24783d9`). I added an auto-clear flag so his state machine could re-lock after the first target (`commits cec899e, 56055e0`). *Result*—Robin integrated the vision stack in one session; on 04 Apr his FSM ran end-to-end against live simulated camera data.

What went well. **STAR (team advocacy—the complaint letter).** *Situation*—three flight days cancelled, two teams had working drones, we had zero flight data [`COMPLAINT_LETTER.md`]. *Task*—communicate the situation to the course organiser without venting. *Action*—I drafted a 1,800-word letter over six iterations (`commits 26e3255, f3401e4, d946838, 5af87d0, 847518e, 4b0fd91`), attached the full chat history as evidence, and explicitly refused to assign blame (“We are not assigning personal blame. We are documenting what happened”). This is the *M17.c non-technical audience* evidence: the reader is a non-engineering course organiser; the artefact is prose with a numeric evidence spine and explicit jargon stripping (“detection pipeline” became “our software that recognises people from the air”). *Result*—the letter opened a constructive clarification thread and produced a written acknowledgement I could cite in the D6 risk register.

What went well. **Method-effectiveness evaluation (M17.d).** I used five communication media on this project: (1) prose report sections, (2) an interactive simulator demo, (3) the `docs/*.blueprint.md` line-map system, (4) teammate-specific READMEs (`passive_watch_2.py` for Robin; `FIELD_QUICK_REF.md`

for no-internet field use; `VISION_PIPELINE_FOR_COLLEAGUE.md` for Robbin), (5) the complaint letter. Judged against *time-to-understanding* per audience: the simulator was highest per hour of author effort (teammate could run it in <1 min), the blueprint system second (new-session ramp from hours to minutes), prose reports lowest (many revision cycles per marker comment). If I repeated the project I would spend less on prose and more on live demos and index artefacts. Where I applied this lesson in-project: I chose a simulator demo over a slide deck for the D5 rehearsal, and got zero clarifying questions against four for the slide version.

Even better if. The piece I am still learning: asking for help early. Most of the support I received arrived after I had already spent days alone on the problem. A mature engineer escalates in hours, not days. My forward commitment is a simple rule—if a bug resists 90 minutes of focused work, I ask a teammate, full stop.

Conclusion. The single-line thesis I would defend on this project is the one I wrote into the team’s complaint letter: “The simulation framework we created ourselves is the only reason this project produced any verifiable engineering output at all.” Everything else—the retrained model, the test ladder, the blueprint system, the teammate tooling, the complaint letter—is the apparatus that kept that thesis honest. The next version of me asks for help sooner, teaches earlier, timeboxes experiments, and builds interfaces before code.